TCP Retransmission and Flow Control (Part II)

CEN 223 – Internet Communication

Assoc. Prof. Dr. Fatih ABUT
Department of Computer Engineering
Çukurova University

What TCP does for you?

- Stream-based in-order delivery
  - Segments are ordered according to sequence numbers
  - Only consecutive bytes are delivered
- Reliability
  - Missing segments are detected (ACK is missing) and retransmitted
- Flow control
  - Receiver is protected against overload (window based)
- Congestion control
  - Network is protected against overload (window based)
  - Protocol tries to fill available capacity
- Connection handling
  - Explicit establishment + teardown
- Full-duplex communication
  - e.g., an ACK can be a data segment at the same time (piggybacking)

TCP Retransmission Strategy

- TCP relies on positive acknowledgements
  - Retransmission on timeout
- Timer associated with each segment as it is sent
- If timer expires before acknowledgement, sender must retransmit

- TCP uses an *adaptive retransmission algorithm* because internet delays are so variable
- *Round trip time* of each connection is recomputed every time an acknowledgment arrives
- Timeout value is adjusted accordingly

Retransmit Timeout Mechanism (1)

- RTO timer value difficult to determine:
  - too high $\Rightarrow$ bad in case of msg-loss!
  - too short $\Rightarrow$ risk of false alarms!
  - General consensus: too short is worse than too long; use conservative estimate
- Calculation: measure RTT (Seg# ... ACK#)
- Original suggestion in RFC 793 (1981)
  - Exponentially Weighed Moving Average (EWMA)
  - $\mathbf{SRTT_{new} = \alpha * SRTT_{old} + (1-\alpha) * RTT_{current}}$
  - $\mathbf{RTO = min(UBOUND, max(LBOUND, \beta * SRTT))}$

SRTT: Smoothed Round Trip Time
α: Smoothing factor (typically 0.8-0.9)
β: Variance factor (typically 2)

Retransmit Timeout Mechanism (2)

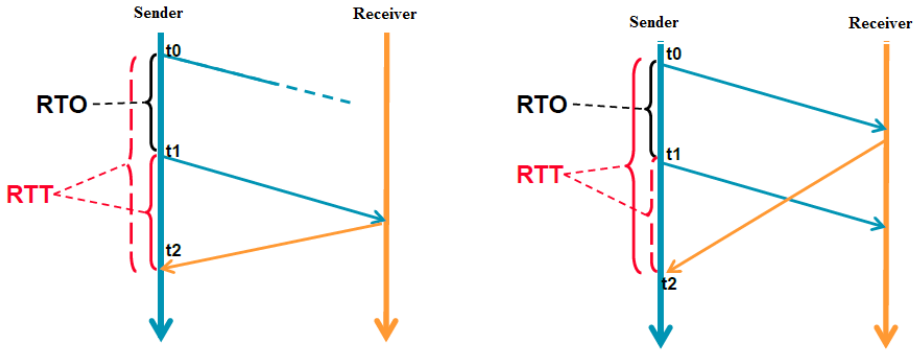
- Depending on variation, this RTO may be too small or too large; thus, final algorithm includes deviation
- Key observation:
  - Original smoothed RTT can't keep up with wide/rapid variations in RTT

- Jacobson/Karel's Retransmission Timeout (1988)

  - $SRTT_{new} = \alpha * SRTT_{old} + (1-\alpha) * RTT_{current}$
  - $SDEV_{new} = \beta * SDEV_{old} + (1 - \beta) * [abs(RTT_{current} - SRTT_{new})]$
  - $RTO_{new} = SRTT_{new} + \gamma * SDEV_{new}$

SDEV: Smoothed deviation
β: Smoothing factor for standard deviation (typically 0.8-0.9)
γ: Adjustment factor (typically 4)

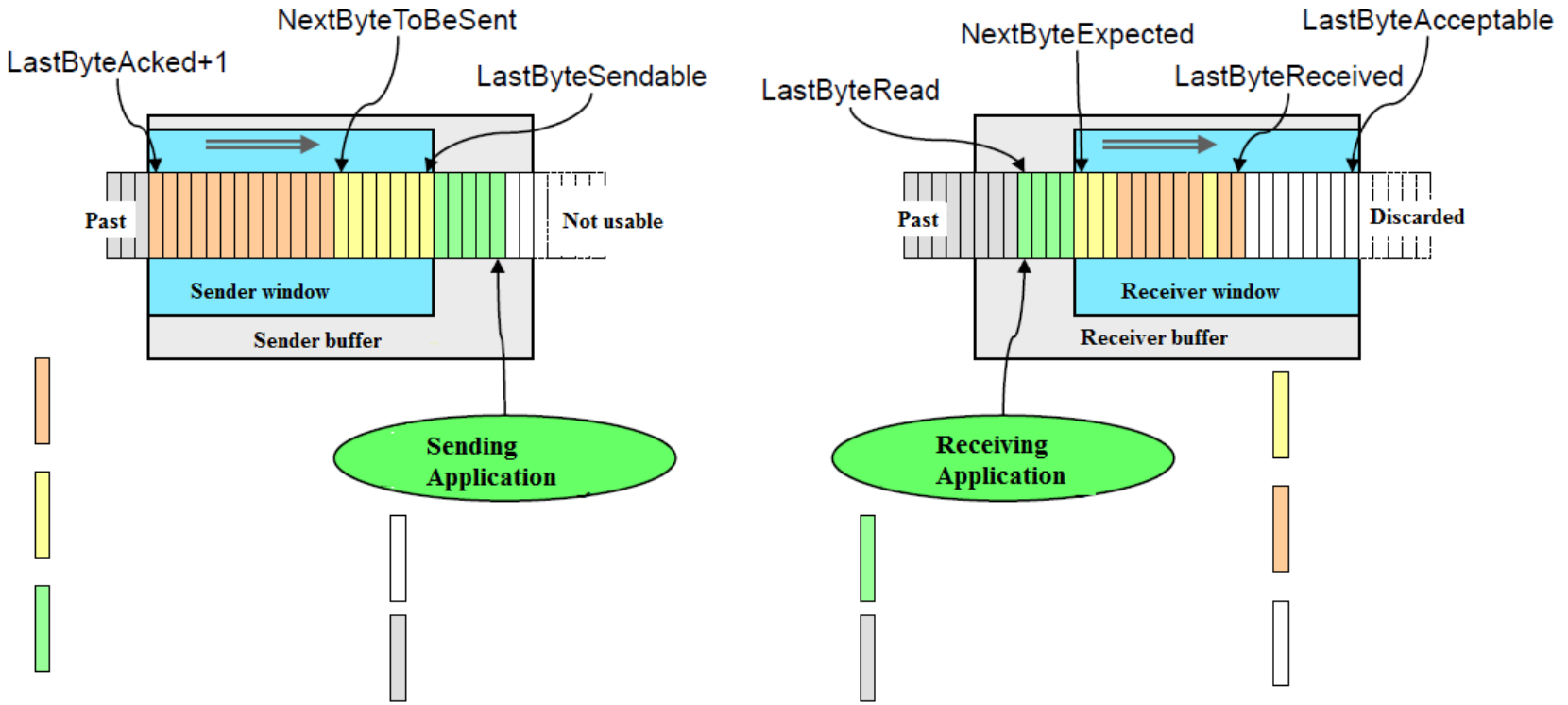
RTO calculation



RTT_{new} = t2-t0 ? RTT_{new} = t2-t1 ?

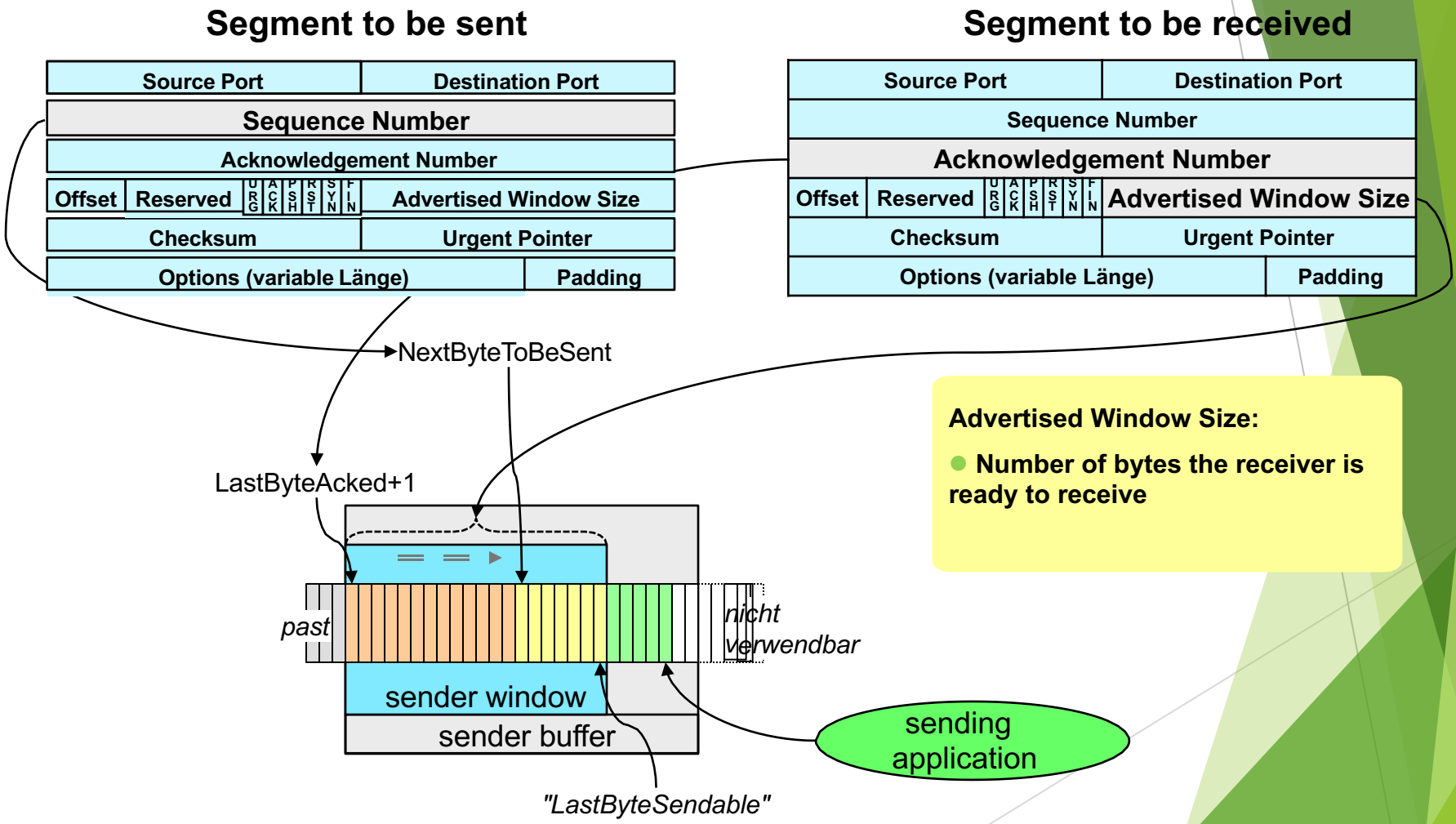
- Problem: retransmission ambiguity
  - Segment #1 sent, no ACK received → segment #1 retransmitted
  - Incoming ACK #2: cannot distinguish whether original or retransmitted segment #1 was ACKed
  - Thus, cannot reliably calculate RTO!
- Solution [Karn/Partridge]: ignore RTT values from retransmits
  - Problem: RTT calculation especially important when loss occurs; sampling theorem suggests that RTT samples should be taken more often
- Solution: Timestamps option
  - Sender writes current time into packet header (option)
  - Receiver reflects value
  - At sender, when ACK arrives, RTT = (current time) - (value carried in option)

Sliding Window Management



- Receiver "grants" credit (receiver window, rwnd)
  - sender restricts sent data with window
- Receiver buffer not specified
  - i.e. receiver may buffer reordered segments (i.e. with gaps)

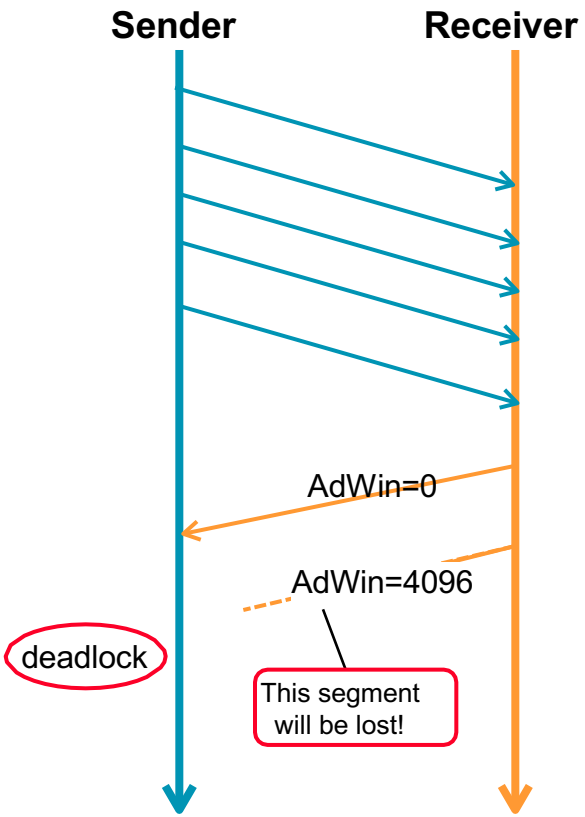
TCP flow control - Segments on the sender side



Advertised Window Size:

- **Number of bytes the receiver is ready to receive**

TCP Flow Control – "Zero advertised window size"



The solution of the problem:

- **Sender uses a persistent timer to periodically send a zero window probe segment as soon as the receiver window is closed.**

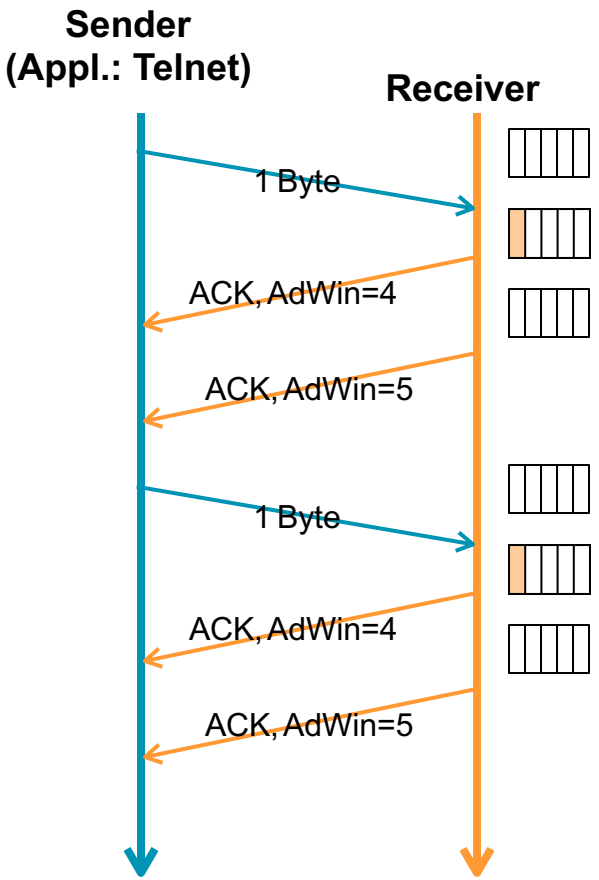
(Algorithm: Exponential back-off algorithm: start value 1.5 seconds, doubling after each Ack until limit, e.g., 60 s, is reached.

→ 1,5 3 6 12 24 48 60 60 60 60 60 ……..)

- **Probe segment does not contain any user data. Specification explicitly allows sample segments to be sent even after the receiver window is closed.**

(Note: As long as the receiver cannot process the sample segment, the ACK contains the sequence number of the last accepted byte)

TCP Flow Control – "Small Packet Problem"



Problem:

- **Sending 2 bytes creates 240 bytes of overhead**
  (2 data segments of 40 bytes each,
  2 acknowledgments of 40 bytes each,
  2 window updates of 40 bytes each)

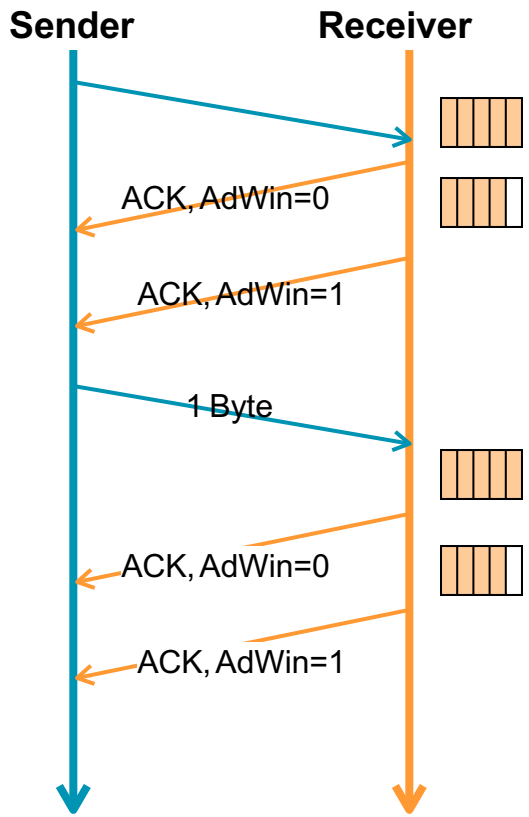
Solutions:

- **Receiver side: delayed Acknowledgement:** Delay of confirmation and window updates by 200 ms (Idea behind: "Piggyback", "Collecting ACKs")
- **Sender side: Nagle's algorithm (RFC 896, 1984):** If data is used byte by byte, only send the first byte, collect bytes and send them in a segment as soon as MSS is reached, or the first byte is confirmed.

  Little influence with IP over LAN, but with WAN.

Problems with interactive applications (Nagle's algorithm leads to jerky work e.g. with X-Window). Therefore Nagle's algorithm can be switched off via sockets.

TCP Flow Control – "Silly Window Syndrome (SWS) RFC 813"



Problem:

- **Byte-wise processing on the receiver side:**

The sender is animated to send small segments.

Solution:

- **Sender side:**
  (1) Avoid sending small amounts of data.
  (2) Nagle's algorithm.
- **Receiver side:**

Window updates only for a larger amount (e.g. if 30% of the receive buffer or 2 MSS are free)

Nagle's Algorithm (1)

- If there is data to send but the window is open less than MSS, then we may want to wait some amount of time before sending the available data
  - If we wait too long, then we hurt interactive applications
  - If we don't wait long enough, then we risk sending a bunch of tiny packets and falling into the silly window syndrome
- The solution is to introduce a timer and to transmit when the timer expires

Nagle's Algorithm (2)

```
When the application produces data to send
    if both the available data and the window ≥ MSS
        send a full segment
    else /* window < MSS */
        if there is unACKed data in flight
            buffer the new data until an ACK arrives
        else
            send all the new data now
```

TCP Acknowledgements (RFC 1122, RFC 2581)

Event	Reaction TCP receiver
Arrival of a directly following segment, no gap, all previous segments have already been confirmed.	**delayed Acknowledgement:** Wait up to 200 ms to see whether a new segment comes, if none comes by then, an ACK must be sent.
Arrival of a directly following segment, no gap, segment waiting for confirmation	**Sending a cumulative ACK:** Confirmation of several segments with a single acknowledgment
Arrival of an out-of-order segment larger than NextByteExpected (there is a gap).	**Sending a duplicate ACK:** Repeated sending of the last ACK (= beginning of the gap)
Arrival of a segment that completely or partially fills a gap (so that part of the byte stream is completed).	**Immediate Acknowledgement:** Immediate sending of an ACK

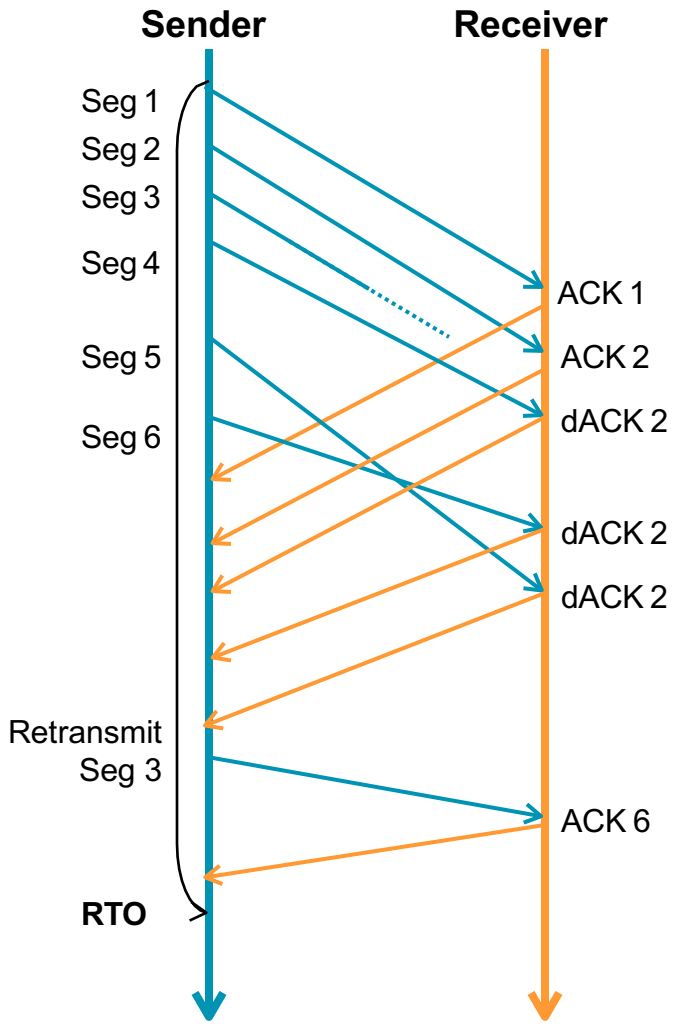
Fast Retransmit (1)

- Coarse timeouts remained a problem, and Fast retransmit was added with TCP Tahoe.
- Since the receiver responds every time a packet arrives, this implies the sender will see duplicate ACKs.
- Basic Idea: *use **duplicate ACKs** to signal lost packet.*

Fast Retransmit

Upon receipt of *three* duplicate ACKs, the TCP Sender retransmits the lost packet.

Fast Retransmit (2)



Problem:

- **Rough setting of the RTO leads to waiting times in the event of packet loss**

Solution:

- **In the event of a triple duplicate ACK, retransmission of the missing package without waiting for the retransmission timer to expire**

New Problem:

- **A lost packet is a sign of network congestion. Therefore, after Fast Retransmit, a reaction to network overload is necessary.**

–> Congestion Control

Note:
Fast Recovery: Algorithm to get data flow after Fast Retransmit
SACK (Selective Acknowledgement, RFC2018): Confirmation of the segments actually received